

Trabalho 5 - Classificacao de padroes usando Adaline

Implementacao didatica em Python baseada no exemplo `adalineFuncoesLogicas.sce` da aula e na organizacao do trabalho anterior: modelo separado da execucao, graficos, resultados auditaveis e testes automatizados.

Objetivos atendidos

- Treinar um Adaline com as duas entradas `s1`, `s2` e o alvo `t` da base B2.
- Plotar o erro quadratico total ao longo das epocas de treinamento.
- Testar a rede treinada, exibindo saida linear, classe prevista, acertos e matriz de confusao.

Executar

No PowerShell, dentro de `Aula05 - Adaline/EXERCICIO`, usando o ambiente virtual existente na raiz do repositório:

```
& '..\..\venv\Scripts\python.exe' -m pip install -r requirements.txt
& '..\..\venv\Scripts\python.exe' main.py
```

Com o ambiente virtual ja ativado:

```
python -m pip install -r requirements.txt
python main.py
```

Requer Python 3.11+ com Tkinter; validado com Python 3.14.6. O comando abre uma janela de **treinamento ao vivo**, inicialmente pausada, com grafo do Adaline, curva do erro e fronteira de separacao. Tkinter acompanha a instalacao oficial do Python para Windows e nao requer um pacote pip adicional.

- **Treinar / Pausar / Continuar**: inicia ou suspende as atualizacoes reais.
- **Passo**: avanca uma amostra ou ate o fim da epoca atual, conforme **Avanco**.
- **Amostra**: destaca o ponto atual e mostra entradas, contribuicoes, saida linear, alvo, erro e pesos antes/depois de cada atualizacao.
- **Epoca**: executa todas as atualizacoes restantes da epoca por quadro, sem pular amostras; e mais rapido para acompanhar as 200 epocas. O grafo mostra a ultima amostra processada no quadro.
- **Intervalo (ms)**: controla o tempo entre quadros, sem alterar a taxa de aprendizado.
- **Reiniciar**: restaura exatamente os pesos iniciais e limpa as curvas.
- **Testar e salvar**: fica disponivel ao terminar as epocas. Fecha a janela de treino, avalia a rede, grava os arquivos e abre os graficos finais, incluindo a matriz de confusao. Feche o grafico final para encerrar.

O grafo mostra a propagacao com os pesos **antes** da atualizacao; a reta mostra os pesos **atuais**, depois da atualizacao. A curva de EQT ganha pontos ao concluir cada epoca. A amostra destacada e numerada dentro do conjunto de treino, nao pela linha do CSV. Os dados de teste nao participam da animacao.

Fechar a janela de treinamento sem **Testar e salvar** cancela a execucao e preserva arquivos de resultados anteriores, mesmo se o treino tiver terminado. Os parametros numericos continuam definidos pelos argumentos de linha de comando. Nao se trata de reproducao de um treino previamente calculado: os pesos sao atualizados durante cada passo, por callbacks Tkinter, sem bloquear a janela.

Para treinar, testar e salvar diretamente, sem animacao nem interface grafica:

```
python main.py --sem-janela
```

Os caminhos padrao da base e dos resultados sao relativos ao script, nao ao diretorio atual do terminal. Os arquivos da base original nao sao modificados.

Experimentos

Por padrao: taxa 0.01, 200 epocas, semente 42, 70% para treino e 30% para teste. A divisao e estratificada: 14 amostras de treino e 6 de teste, com ambas as classes nos dois conjuntos. As amostras selecionadas sao apresentadas em ordem de linha do CSV, sem embaralhar a cada epoca, seguindo o exemplo sequencial da aula.

```
python main.py --taxa 0.01 --epocas 200 --fracao-teste 0.3 --semente 42
```

Para reproduzir o procedimento da aula, treinando com **todas as 20 amostras** da B2 e conferindo a rede sobre essas mesmas amostras:

```
python main.py --fracao-teste 0 --saida resultados_base_completa
```

Esse segundo modo e identificado como **reavaliacao do treino**, e nao como teste independente. Para selecionar outra base ou preservar resultados de experimentos:

```
python main.py --base Basedados_B2.csv --saida resultados_experimento --sem-janela
```

Os arquivos de mesmo nome na pasta de saida sao substituidos em cada execucao. Consulte `python main.py --help` para os parametros disponiveis.

Dados e algoritmo

O CSV tem cabecalho `s1,s2,t`. Os campos numericos entre aspas usam virgula decimal: `"2,215"` e convertido em `2.215` pelo leitor, sem confundir a virgula decimal com o separador de colunas. Sao 20 amostras, 10 de cada classe.

As entradas sao **contínuas**, preservadas nas unidades originais, sem normalizacao ou conversao para -1/+1. Somente os alvos e as classes previstas sao bipolares. Isso e diferente da codificacao dos pixels do trabalho anterior.

O Adaline tem dois pesos e um bias. Durante o treino, a ativacao e linear:

$$u = w_1 s_1 + w_2 s_2 + b, \quad e = t - u$$

A regra delta atualiza cada peso e o bias para **todas** as amostras:

$$w_j \leftarrow w_j + \alpha e s_j, \quad b \leftarrow b + \alpha e$$

Diferentemente do perceptron, o erro usa a saida linear, nao a classe obtida por degrau. Mesmo um padrao com classe correta pode ajustar os pesos. Os pesos e o bias sao inicializados uniformemente entre -0.5 e 0.5 com semente fixa. A parada ocorre exatamente apos o numero de epocas solicitado.

Somente na classificacao aplica-se o degrau com limiar zero:

$$\hat{t} = \begin{cases} +1, & u \geq 0 \\ -1, & u < 0 \end{cases}$$

A fronteira de decisao e a reta $w_1 s_1 + w_2 s_2 + b = 0$. As saidas lineares nao sao probabilidades. O teste nao modifica pesos nem bias.

Erro quadratico total

O erro quadratico total (EQT) e uma **soma**, nao uma media e nao e dividido por 2:

$$EQT = \sum_{i=1}^N (t_i - u_i)^2$$

O grafico e o historico registram duas medidas, calculadas somente no treino:

- **online_sse**: soma dos erros antes de cada atualizacao dentro da epoca, exatamente como no exemplo Scilab. Os pesos mudam entre as amostras.
- **sse**: soma recalculada sobre todo o treino com os pesos fixos ao fim da epoca. Permite conferir diretamente o EQT do modelo salvo.

O erro inicial e exibido no terminal e salvo no JSON. A curva ao vivo de pesos fixos inclui esse valor na epoca 0; o historico exportado e o grafico final mostram as epocas 1 ate o total solicitado. O EQT nao precisa chegar a zero para classificar corretamente. Na regra delta sequencial ele pode oscilar, e taxas altas podem provocar divergencia. Nesse caso, reduza **--taxa**.

Resultados e auditoria

Na pasta **resultados**, a execucao gera:

- **resultado.png**: curvas de EQT, dados com fronteira e matriz de confusao.
- **historico.csv**: epoca, os dois erros, pesos e bias ao final de cada epoca.
- **predicoes.csv**: linha original do CSV, entradas, alvo, saida linear, classe prevista e indicador de acerto para cada amostra avaliada.
- **modelo.json**: pesos iniciais/finais, bias, parametros, EQT, acuracias, matriz de confusao e linhas usadas em treino e teste.

As linhas do CSV original sao numeradas a partir de 1, incluindo o cabecalho. Nos arquivos gerados, os decimais usam ponto. Os pesos salvos permitem conferir qualquer previsao com $u = w1*s1 + w2*s2 + bias$ e o degra de limiar zero.

Execucao verificada com os parametros padrao:

Medida	Resultado
Treino / teste	14 / 6 amostras
EQT inicial no treino	25.207205
EQT final no treino	2.540616
EQT acumulado na ultima epoca	2.642631
Pesos finais	-0.66088761, -0.63405071
Bias final	1.885494
Acuracia no treino	100%
Acuracia no teste reservado	100% (6/6)

Matriz de confusao do teste, com linhas reais e colunas previstas na ordem -1,+1:

```
3  0
0  3
```

Limitacao: seis amostras de teste sao poucas. Esse resultado descreve uma divisao especifica e nao garante desempenho em novos dados. Nao escolha taxa, epocas ou semente com base no teste se quiser preservar uma avaliacao independente; para ajuste de parametros, use um conjunto de validacao separado.

Testes automatizados

```
python -m unittest discover -s . -p test_adaline.py -v
```

Os testes verificam a regra delta com calculo manual, leitura das virgulas decimais, separacao sem sobreposicao, reducao do EQT, limiar zero, preservacao dos pesos no teste, reproducao das previsoes salvas e geracao de imagem nao vazia. Tambem verificam o modo de reavaliacao da base completa e a equivalencia entre `fit()` e o iterador por amostra `iter_fit()`, ambos usando a mesma regra delta. O teste `TrainingWindowTests` abre uma janela brevemente e verifica pausa, reinicio, avanco, conclusao, cancelamento e renderizacao. Requer uma sessao grafica. Para testar somente o modelo e as exportacoes, sem janela:

```
python -m unittest test_adaline.AdalineTests -v
```